

COMP3320/6464 Week 5, Lecture 2

Rhys Hawkins

Semester 2, 2024

Overview

Today

- Introduction to SIMD
- Compiler auto-vectorization
- SIMD Intrinsics (2nd half)

Types of Parallelism

- Task Parallelism: Executing different threads of control in parallel
- Data Parallelism: Performing the same operation on different pieces of data
 - e.g. summing two vectors element by element
 - **SIMD** $\equiv$ Single Instruction Multiple Data
- Instruction Level Parallelism: Multiple instructions are concurrently executed
 - Superscalar - Multiple functional units
 - Out-of-order execution and pipelining

SIMD - Multiple operations are concurrent, while instructions are the same

Flynn's Taxonomy

Mike Flynn, “Very High-Speed Computing Systems”, Proceedings of IEEE, 1966

- SISD: Single instruction single data
- **SIMD**: Single instruction multiple data
- MIMD: Multiple instruction multiple data
- SIMT: Single instruction multiple thread (e.g. GPU)

History of SIMD - Vector Processors

- Instructions on a vector processor operate on vectors rather than scalar values
- Has vector registers where vectors can be loaded from or stored to
- Vectors may be of variable length
 - i.e. vector registers must support variable vector lengths
- Data elements to be loaded into a vector register may not be contiguous in memory
 - i.e. support for strides or distances between two elements of a vector
- Cray-I used vector processors
 - Clocked at 80 MHz in Los Alamos National Lab, 1976
 - Introduced CPU registers for SIMD vector operations
 - 240 MFLOPS when SIMD operations utilized effectively
- Primary disadvantage: Works well only if parallelism is regular
- Superseded by contemporary scalar processors with support for vector operations i.e. SIMD extensions

Scalar versus Vector processor

Adding two vectors of length 10, $C = A + B$

Loop 10 iterations

Read i_{th} instruction and decode

Fetch the $A[i]$ element

Fetch the $B[i]$ element

Add $A[i] + B[i]$

Store result in $C[i]$

Increment i till $i=10$

General Processing

Read instruction and decode

Fetch the all 10 elements of $A[]$

Fetch the all 10 elements of $B[]$

Add $A[] + B[]$

Store result in $C[]$

Vector Processing

From <https://binaryterms.com/vector-processing.html>

SIMD Extensions

Extensive use of SIMD extensions in Contemporary Hardware:

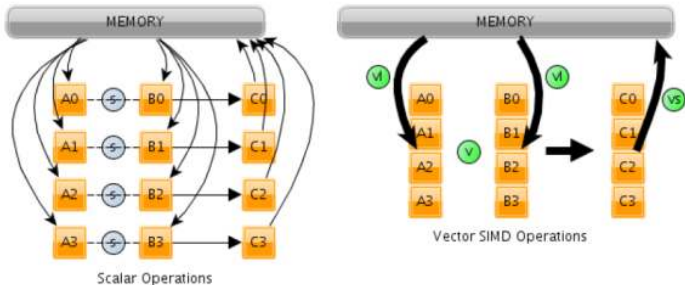
- Complex Instruction Set Computers (CISC)
 - Intel MMX: 64-bit wide registers - first widely used SIMD instruction set on the desktop computer in 1996
 - Intel Streaming SIMD Extensions (SSE): 128-bit wide XMM registers
 - Intel Advanced Vector Extensions (AVX): 256-bit wide YMM registers
 - Intel/AMD Fused Multiply Add (FMA 3/4): 256-bit $a \times b + c$ operations
 - Intel AVX-512: 512-bit wide ZMM registers
- Reduced Instruction Set Computers (RISC)
 - SPARC64 V8 (HPC-ACE): 128-bit registers
 - PowerPC A2 (Altivec, VSX): 128-bit registers
 - ARMv7, ARMv8 (NEON): 64-bit and 128-bit registers
- Similar architecture: Single Instruction Multiple Thread (SIMT) used in GPUs

SIMD Processing - Vector addition

```
void VectorAdd(float *a, float *b, float *c, size_t size)
{
    size_t i;
    for (i = 0; i < size; i++)
    {
        c[i] = a[i] + b[i];
    }
}
```

- Assume arrays a and b contain 32 bit floats
- No dependencies between operations
- Note: Array's A and B may not be contiguously allocated
- How can this operation be parallelized ?

SIMD Processing - Vector addition



- Scalar: 8 loads + 4 scalar adds + 4 stores = 16 ops
- Vector: 2 loads + 1 vector add + 1 store = 4 ops
- 4x Speedup

Fundamental idea: Perform multiple operations using single instructions on multiple data items concurrently Advantages:

- Performance improvement
- Fewer instructions - reduced code size, maximization of data bandwidth
- Automatic Parallelization by compiler for vectorizable code

Intel SSE



- 8 (in 32-bit mode) or 16 (in 64-bit mode) 128-bit XMM Registers
- XMM0 - XMM15
- 8, 16, 32, 64-bit Integers
- 32-bit SP and 64-bit DP

Intel AVX

255 ... 128	127 ... 0
0.0	xmm0
ymm0	
0.0	xmm1
ymm1	
...	...
...	
0.0	xmm14
ymm14	
0.0	xmm15
ymm15	

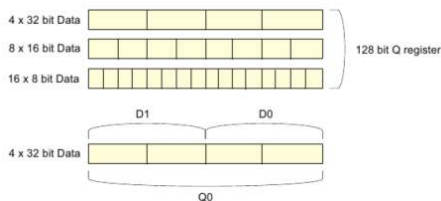
- Introduces new three-operand instructions i.e. one destination, two source operands
- Previously, SSE instructions had the form $a = a + b$
- With AVX, the source operands are preserved i.e. $c = a + b$

- Intel Advanced Vector Extensions (2008)
 - extended vectors to 256 bits
- AVX2 (2013)
 - Expands most integer SSE and AVX instructions to 256 bits
- Intel FMA3 (2013)
 - Fused multiply-add introduced in Haswell
- 8 or 16 256-bit YMM Registers
- YMM0 - YMM15
- SSE instructions operate on lower half of YMM registers

Intel AVX 512

- ZMM0 - ZMM31 512 bit registers
- Same width as a cache line
- Also means that XMM and YMM are extended to 0 - 31
- The use of “heavy” AVX 512 instructions incurs a performance penalty
 - Upon the first use of a “heavy” instruction, it is executed 4x as slow
 - Repeat use of these instructions results in a lowering of the CPU clock rate
 - Mixed instructions of AVX 512 and other SIMD instructions *probably* would run faster using straight AVX 256 instructions.
 - Good timing is required to validate (you can't use cycle counting)

ARM NEON



- ARM Advanced SIMD (NEON)
- ARM Advanced SIMDv2
 - Support for fused multiply-add
 - Support for half-precision extension
- Separate register file
- 32 64-bit Registers
- Shared by VFPv3/VFPv4 instructions
- NEON register views:
 - D0-D31: 32 64-bit Double-word
 - Q0-Q15: 16 128-bit Quad-word
- 8, 16, 32, 64-bit Integers ARMv7:
- 32-bit SP Floating-point
- ARMv8: 32-bit SP and 64-bit DP

SIMD Instruction Types

- Data Movement: Loading, storing vectors between main memory and SIMD registers
- Arithmetic operations: Addition, subtraction, multiplication, division, absolute difference, maximum, minimum, saturation arithmetic, square root, multiply-accumulate, multiply-subtract, halving-subtract, folding maximum and minimum
- Logical operations: Bitwise AND, OR, NOT operations and their combinations
- Data value comparisons: $=$, $<=$, $<$, $>=$, $>$
- Pack, Unpack, Shuffle: Initializing vectors from bit patterns, rearranging bits based on a control mask
- Conversion: Between floating-point and integer data types using saturation arithmetic
- Bit Shift: Often used to do integer arithmetic such as division and multiplication
- Other: Cache specific operations, casting, bit insert, cache line flush, data prefetch, execution pause etc

How to use SIMD operations

In order of increasing difficulty

- Compiler auto-vectorization: Requires a compiler with vectorizing capabilities. Least time consuming. Performance variable and entirely dependent on compiler quality.
- Compiler intrinsic functions: Almost one-to-one mapping to assembly instructions, without having to deal with register allocations, instruction scheduling, type checking and call stack maintenance.
- Inline assembly: Writing assembly instructions directly into higher level code
- Low-level assembly: Best approach for high performance. Most time consuming.

Compiler Auto-vectorization

- Requires a vectorizing compiler e.g. gcc, icc/icx
- On Gadi
 - `module load intel-compiler`
 - `module load gcc/12.2.0`
- Loop unrolling combined with the generation of packed SIMD instructions
- GCC enables vectorization with -O3
 - Enabled with -O2 on Intel compilers
 - For x86_64, default is SSE2 instructions
 - Instruction set specified by -msse2 (-msse4.1 -mavx) for Intel systems
 - Enabled with -mfpu=neon on ARM systems
- Reports from vectorization process
 - gcc: `-fopt-info-vec-optimized`
 - intel: `-vec-report5`

Compiler Auto-vectorization - Loops

What kind of loops are good candidates for auto-vectorization?

- Countable: The loop trip count must be known at entry to the loop at runtime
- Single entry and single exit: No break
- Straight-line code: It is not possible for different iterations to have different flow-control (must not branch). If statements allowed if they can be implemented as masked assignments
- The innermost loop of a nest: Possible loop interchange in previous optimization phases
- No function calls: Some intrinsic math functions allowed (sin, log, pow etc)
- No aliasing: Pointers to vector arguments should be declared with keyword restrict which guarantees that no aliases exist for them

Compiler Auto-vectorization - Obstacles

- Non-contiguous memory accesses: Four consecutive floats (ints) can be loaded directly. If there is a stride, they have to be loaded separately using multiple instructions.
- Non-aligned data structures: May result in multiple load instructions
- Data dependencies: RAW, WAR, WAW

Data dependencies

- Read after Write (RAW)

- 1 $R2 = R1 + R3$

- 2 $R4 = R2 + R3$ ($R2$ results from 1 must have been saved before 2)

- Write after Read (WAR)

- 1 $R4 = R1 + R5$

- 2 $R5 = R1 + R2$ ($R5$ cannot be stored before 1 is finished)

- Write after Write (WAW)

- 1 $R2 = R1 + R3$

- 2 $R2 = R4 + R5$ (write back of 2 must be delayed until 1 is finished)

Compiler Auto-vectorization - Example 1

- SAXPY
- $Y = X + Y$ in Single Precision
- Here 'a' is scalar constant and X,Y are SP vectors
- Used in BLAS (Basic Linear Algebra Subprograms) library

```
void saxpy(int n, float a, float* __restrict__ X, float* __restrict__ Y)
{
    int i;
    for (i=0; i<n; i++)
        Y[i] = a*X[i] + Y[i];
}
```

Compiler Auto-vectorization - Example 1

Compile using vectorization options:

```
> gcc -c -O3 -fopt-info-vec-optimized saxpy.c  
saxpy.c:21:21: optimized: loop vectorized using 16 byte vectors
```

For much more output of what could and could not be vectorized

```
> gcc -c -O3 -fopt-info-vec-all saxpy.c  
...
```

Compiler Auto-vectorization - Verifying with Compiler Output

```
gcc -c -O3 -fopt-info-vec-optimized saxpy.c
```

```
saxpy.c:21:21: optimized: loop vectorized using 16 byte vectors  
saxpy.c:21:21: optimized: loop vectorized using 8 byte vectors  
saxpy.c:21:21: optimized: loop vectorized using 16 byte vectors
```

- 8 byte vectors XMM
- 16 byte vectors YMM
- 32 byte vectors ZMM

Compiler Auto-vectorization - Verifying with Assembler Output

```
gcc -S -O3 -fverbose-asm -masm=intel -o saxpy.s saxpy.c  
less saxpy.s
```

```
.L4:  
# test_saxpy.c:22:      Y[i] = a*X[i] + Y[i];  
    movups  xmm1, XMMWORD PTR [rcx+rax]      # vect__5.12, MEM <vector(4) float>  
# test_saxpy.c:22:      Y[i] = a*X[i] + Y[i];  
    movups  xmm3, XMMWORD PTR [rdx+rax]      # tmp148, MEM <vector(4) float> [(fl  
# test_saxpy.c:22:      Y[i] = a*X[i] + Y[i];  
    mulps   xmm1, xmm2      # vect__5.12, vect_cst__51  
# test_saxpy.c:22:      Y[i] = a*X[i] + Y[i];  
    addps   xmm1, xmm3      # vect__8.16, tmp148
```

Note the use of xmm registers, and commands with the suffix “ps”

- ps - packed single
- pd - packed double
- ss - scalar single (non-SIMD)
- sd - scalar double (non-SIMD)
- xmm registers are used for all floating point operations on x86_64 architecture

Compiler Auto-vectorization - Verifying with Godbolt

<https://godbolt.org>

The screenshot displays the Compiler Explorer interface with the following details:

- Language:** C
- Compiler:** x86_64 gcc 12.2
- Options:** -O3 -mavx2
- C Code:**

```

1 int sum(int a, int x, int y) {
2     int i = x;
3     while (i <= y) {
4         sum[i] = a[i] * i;
5         i++;
6     }
7     return sum[y];
8 }

```
- Assembly Output:**

```

1 sum(int a, float x, float y)
2     mov     rax, rdi
3     test    rdi, rdi
4     jle     .L4
5     mov     rcx, [rdi+1]
6     cmp     rcx, 1
7     jbe     .L5
8     mov     rdi, rcx
9     vfmulsdsscaleq    rcx, rcx
10
11     mov     rcx, rcx
12     shr     rcx, 1
13     sal     rcx, 1
14
15     vmulps  rcx, rcx, xmmword ptr [rdi+rcx]
16     vstps   rcx, rcx, xmmword ptr [rdi+rcx]
17     vmovaps xmmword ptr [rdi+rcx], rcx
18     add     rcx, 16
19     cmp     rcx, y
20     jns     .L4
21     and     rcx, -8
22     mov     rcx, rcx
23     cmp     rcx, x
24     jle     .L5
25     vstps   rcx, rcx, xmmword ptr [rdi+rcx]
26
27 .L4:
28     mov     rcx, rcx
29     shr     rcx, 4
30     mov     rcx, [rdi+rcx]
31     cmp     rcx, 1
32     jbe     .L5
33     mov     rcx, [rdi+rcx]
34     vfmulps rcx, rcx, xmmword ptr [rdi+rcx]
35     vstps   rcx, rcx, xmmword ptr [rdi+rcx]

```


Compiler Auto-vectorization - Verifying with Godbolt

<https://godbolt.org>

SSE instructions (using xmm registers with “ps” suffix)

```
movups  xmm1, XMMWORD PTR [rcx+rax]
movups  xmm3, XMMWORD PTR [rdx+rax]
mulps   xmm1, xmm2
addps   xmm1, xmm3
movups  XMMWORD PTR [rdx+rax], xmm1
```

AVX instructions (using ymm registers)

```
.L4:
vmulps  ymm1, ymm2, YMMWORD PTR [rcx+rax]
vaddps  ymm1, ymm1, YMMWORD PTR [rdx+rax]
vmovups YMMWORD PTR [rdx+rax], ymm1
```

Compiler Auto-vectorization - Example 1

With gcc 12.2.0 (make sure you use `module load gcc/12.2.0`)

Optimization	Time (μ s)	Speed Up	Register Use
None	29185	1	xmm (addss)
-O3	1633	18	xmm (addps)
-mavx2	1531	19	ymm
-mavx512f *	1254	23	zmm

* with extra flag `-mprefer-vector-width=512`

Compiler Auto-vectorization - Example 1

With `icc/x` (module load intel-compiler)

Optimization	Time (μ s)	Speed Up	Register Use
-O0	15191	1	xmm (addss)
-O2	2297	18	xmm (addps)
-xCORE_AVX	804	19	ymm
-xCORE_AVX512 *	601	23	zmm

* with extra flag `-qopt-zmm-usage=high`

Compiler Auto-vectorization - Example 2

- Demonstrates the cost of calling functions
- Also a loop carried dependency

```
float loop1(float *A)
{
    float s = 1.0;
    for (int i = 0; i < N; i++) {
        s *= exp(A[i]);
    }
    return s;
}
```

```
float loop2(float *A)
{
    float s = 0.0;
    for (int i = 0; i < N; i++) {
        s += A[i];
    }
    return exp(s);
}
```

Compiler Auto-vectorization - Example 2

With gcc 12.2.0 (make sure you use `module load gcc/12.2.0`)

Optimization	Loop 1 (μ s)	Loop 2 (μ s)
None	104061	16906
-O3	88363	7373
-mavx2	93085	7371
-mavx512f *	-	-

* with extra flag `-mprefer-vector-width=512`

- Unable to vectorize with AVX512
- Poor performance and failure to use AVX512 is due to loop carried dependency

Compiler Auto-vectorization - Example 2

- Recall that floating point math is not associative, i.e.
 $(a + b) + c \neq a + (b + c)$
- gcc will not use separate accumulators to optimize the loop carried dependency (by default)
- We can allow gcc to do this in different ways (in order of increasing numerical approximations)
 - -fassociative-math
 - -funsafe-math-optimizations
 - -ffast-math

Compiler Auto-vectorization - Example 2

With gcc 12.2.0 (make sure you use `module load gcc/12.2.0`)

Optimization	Loop 1 (μ s)	Loop 2 (μ s)
None	104061	16906
-O3	88363	7373
-mavx2	93085	7371
-mavx512f *	-	-
-O3 †	91601	1868
-mavx2 †	95058	933
-mavx512f * †	94707	503

* with extra flag `-mprefer-vector-width=512`

† with `-funsafe-math-optimizations`

- Significant improvement in performance
- Accuracy of results can be affected.

Compiler Auto-vectorization - Example 2

With `icc/x` (module `load intel-compiler`)

Optimization	Loop 1 (μ s)	Loop 2 (μ s)
-O0	60330	17081
-O2	55765	571
-xCORE_AVX	11223	371
-xCORE_AVX512 *	6899	263

* with extra flag `-qopt-zmm-usage=high`

- Intel compilers by default use “unsafe” math optimizations
- Note also the speed up of the `exp` version of the loop.
- Intel libraries have (some) vectorized math functions

Auto-vectorization - Summary

- Compiler auto-vectorization works well for simple loops
- A few simple rules to improve auto-vectorization
 - Fixed loop lengths at runtime
 - Prefer simple operations (Branching and function calls interfere)
 - Remove loop dependencies if possible
 - Remove function calls if possible
- Compiler flags to turn off strict math rules may be required to obtain best performance.
- This impacts accuracy and applies to the whole module/code base.